

ACI PİRAMİDİ

[Kaynak](#)

İÇERİK

1. Hash Values (Trivial)
2. IP Address (Kolay Seviye) - Pyramid of Pain ve Ağ Gizlenme Teknikleri
3. Alan Adları - Domain Names (Basit Seviye)
4. Host Artifacts (Uç Nokta Kalıntıları) - Orta Seviye
5. Ağ Kalıntıları (Network Artifacts) - Can Sıkıcı (Annoying) Seviye
6. Araçlar (Tools) - Zorlu (Challenging) Seviye

Hash Values (Trivial)

Kriptografi ve siber güvenlik dünyasında bir **Hash** (Özet Değeri), değişken uzunluktaki herhangi bir verinin, matematiksel bir **Hashing Algorithm** (Özetleme Algoritması) işleminden geçirilerek sabit uzunlukta (fixed-length) ve benzersiz (unique) bir sayısal değere dönüştürülmüş halidir. Microsoft'un da tanımladığı üzere, bu değer ilgili verinin dijital parmak izidir. Bütünlük (integrity) doğrulaması ve zararlı yazılım (malware) tespiti süreçlerinde temel bir yapı taşıdır.

Bir hash algoritmasının kriptografik olarak güvenli kabul edilebilmesi için en temel kural, iki farklı dosyanın veya veri setinin asla aynı hash değerini üretmemesidir. Eğer üretirse, bu duruma **Hash Collision** (Hash Çakışması - Farklı iki girdinin aynı özet değerine sahip olması) adı verilir ve algoritmanın artık güvenilir olmadığı anlamına gelir.

Endüstride Yaygın Kullanılan Hash Algoritmalarının Evrimi ve Zafiyetleri

Güvenlik mimarilerinde tarihsel süreç boyunca çeşitli hash algoritmaları kullanılmıştır. Bu algoritmaların teknik derinlikleri ve güncel durumları şu şekildedir:

MD5 (Message Digest 5): 1992 yılında Ron Rivest tarafından tasarlanan bu algoritma, girdiyi 128-bit uzunluğunda bir hash değerine dönüştürür. Günümüz standartlarında MD5, kriptografik olarak güvenli kabul EDİLMEZ. IETF'nin (Internet Engineering Task Force) 2011'de yayımladığı RFC 6151 dokümanında detaylandırıldığı üzere, MD5 algoritması **Hash Collision** saldırılarına karşı son derece kırılgandır. Bu zafiyet, bir saldırganın zararlı bir dosyayı, zararsız bir dosyayla aynı MD5 hash değerine sahip olacak şekilde manipüle edebilmesine olanak tanır.

SHA-1 (Secure Hash Algorithm 1): 1995 yılında NSA (Amerikan Ulusal Güvenlik Ajansı) tarafından icat edilmiştir. Veriyi işleyerek 160-bit uzunluğunda, 40 haneli onaltılık

(hexadecimal) bir dize üretir. NIST (Ulusal Standartlar ve Teknoloji Enstitüsü), **Brute-Force** (Kaba Kuvvet - Tüm olası kombinasyonların sistematik olarak denenmesi) saldırılarına karşı zayıf olduğu gerekçesiyle 2011 yılında SHA-1'in kullanımını "kullanımdan kaldırılmış (deprecated)" olarak işaretlemiş ve 2013'ün sonunda dijital imzalarda kullanımını tamamen yasaklamıştır.

SHA-2 (Secure Hash Algorithm 2) ve SHA-256: SHA-1'in yerini alması için 2001 yılında NIST ve NSA tarafından ortaklaşa geliştirilmiştir. Ailenin en yaygın varyantı olan SHA-256, veriyi 256-bit uzunluğunda ve 64 haneli onaltılık bir değere dönüştürür. Modern sistemlerde bütünlük doğrulaması için mevcut endüstri standardıdır ve SHA-1'den SHA-2 veya SHA-3 ailelerine geçiş NIST tarafından şiddetle tavsiye edilmektedir.

Tehdit Avcılığı (Threat Hunting) ve Olay Müdahalesinde Hash Kullanımı

Siber güvenlik uzmanları ve DFIR (Digital Forensics and Incident Response) ekipleri, şüpheli bir dosyanın veya zararlı yazılım numunesinin kimliğini tekil olarak belirlemek için hash değerlerini kullanırlar. Güvenlik raporlarında (örneğin The DFIR Report veya Trellix Threat Research analizlerinde) raporun sonunda genellikle **IOC** (Indicators of Compromise - Uzlaşma Göstergeleri) listesi verilir. Bu liste, saldırganın sistemde bıraktığı veya kullandığı zararlı dosyaların hash değerlerini içerir.

DETECTION	DETAILS	RELATIONS	BEHAVIOR	COMMUNITY
Antiy-AVL	① Trojan.Generic.ASSu.19EC8	CAT-QuickHeal	① RTF.Trojan.Agent.40329	
Comodo	① Malware.17uob1a9vm9d	ESET-NOD32	① Win32/Filecoder.WannaCryptor.D	
Gridinsoft	① Ransom.U.Ransom.oa	Ikarus	① Trojan.Win32.Filecoder	
Lionix	① Trojan.MSOffice.Generic.4lc	McAfee	① RTF/Wannacry.a	
McAfee-GW-Edition	① RTF/Wannacry.a	Microsoft	① Ransom:Win32/WannaCrypt.Alsrm	
Symantec	① Trojan.Gen.NPE.2	Tencent	① Win32.Trojan.Filecoder.Dvzt	
TrendMicro	① TROJ_RANSOMNOTE.RTF	TrendMicro-HouseCall	① TROJ_RANSOMNOTE.RTF	

Bu hash değerleri elde edildiğinde, dosyanın daha önce analiz edilip edilmediğini görmek için **VirusTotal** veya **MetaDefender Cloud - OPSWAT** gibi çevrimiçi istihbarat motorlarında (OSINT) taramalar yapılır. Örneğin, bir hash aratıldığında VirusTotal bu hash'in `m_croetian.wnry` adlı bir Ransomware (Fidye Yazılımı) dosyasına ait olduğunu saniyeler içinde doğrulayabilir. Bu, savunma tarafı için inanılmaz hızlı bir tespit mekanizmasıdır.

Overview

Static Analysis

Community

E325988F68D327743926EA317ABB9882F347...

Sanitized version

Threat name: Trojan/WcrylyBhUK2kw

Cast your vote on this file: 0

Metascan

Threats detected

08 /34
ENGINES

Get full report

Upgrade limits

Sandbox Threat Score

No dynamic analysis performed

00 /10

View dynamic analysis

Sandbox documentation

Community Insight

User votes

0 %

View leaderboards

Check out our community

ÖNEMLİ: Hash değerleri mükemmel birer IOC olsalar da, savunma perspektifinden bakıldığında oldukça "kırılgan" göstergelerdir. Bir dosyanın içerisindeki tek bir bit'in (0'ın 1 yapılması) veya dosyanın sonuna anlamsız tek bir karakterin eklenmesi, Avalanche Effect (Çığ Etkisi) yaratarak dosyanın üreteceği hash değerini tamamen ve köklü bir şekilde değiştirir.

Bir Saldırgan Perspektifinden Hash Manipülasyonu (Evasion)

Saldırganların bilinen bir zararlı yazılımı (örneğin bilindik bir ransomware) antivirüslerin ve EDR (Endpoint Detection and Response) çözümlerinin hash tabanlı kara listelerinden (blacklist) nasıl kaçırdığını (evasion) anlamak, analist olmanın temel şartıdır.

Aşağıdaki laboratuvar ortamında, standart bir dosyanın hash değerinin ne kadar basit bir komutla tamamen değiştirilebileceğini inceliyoruz.

İlk olarak, hedef dosyanın (bu senaryoda bir MSI kurulum dosyası) orijinal hash değerini alıyoruz:

```
PS C:\Users\THM\Downloads> Get-FileHash .\OpenVPN_2.5.1_I601_amd64.msi -Algorithm MD5
```

Bu komutta PowerShell'in yerleşik Get-FileHash cmdlet'ini kullanıyoruz. İşlem yapılacak dosyanın yolu **.\OpenVPN_2.5.1_I601_amd64.msi** olarak belirtilmiştir. -Algorithm parametresi ile varsayılan SHA-256 değeri yerine **MD5** algoritmasını seçtiğimizi komut satırına açıkça iletiyoruz.

```
PS C:\Users\THM\Downloads> Get-FileHash .\OpenVPN_2.5.1_I601_amd64.msi -Algorithm MD5
Algorithm Hash Path
-----
MD5 D1A008E3A606F24590A02B853E955CF7 C:\Users\THM\Downloads\OpenVPN_2.5.1_I601_amd64.msi
```

Çıktıda, orijinal dosyanın MD5 hash değerinin D1A008E3A606F24590A02B853E955CF7 olduğunu görüyoruz. Eğer bu dosya zararlı bir yazılım olsaydı, savunma araçları bu hash değerini tanıyıp engellerdi.

Şimdi bir saldırganın uygulayacağı en basit tahrifat yöntemini (file modification) simüle ediyoruz:

```
PS C:\Users\THM\Downloads> echo "AppendTheHash" >> .\OpenVPN_2.5.1_I601_amd64.msi
```

Bu adım kritik bir operasyondur. `echo` komutu ile "AppendTheHash" şeklinde tamamen anlamsız (veya rastgele) bir metin dizisi oluşturuyoruz. Burada kullanılan `>>` (append) operatörü hayati önem taşır. Eğer tekil büyüktür `>` (overwrite) kullansaydık dosyanın içeriğini tamamen silip üstüne yazardık ve dosya bozulurdu. Ancak `>>` operatörü, dosyayı bozmadan (binary yapısının en sonuna) bu metni ekler. Dosyanın boyutu sadece birkaç byte artar ve dosyanın temel işlevi (çalışabilirliği) çoğu zaman etkilenmez.

Dosyanın içeriği (bit dizilimi) artık değiştiği için, bütünlük kırılmıştır. Yeni hash değerini kontrol ediyoruz:

```
PS C:\Users\THM\Downloads> Get-FileHash .\OpenVPN_2.5.1_I601_amd64.msi -Algorithm MD5
```

Algorithm	Hash	Path
MD5	9D52B46F5DE41B73418F8E0DACEC5E9F	C:\Users\THM\Downloads\OpenVPN_2.5.1_I601_amd64.msi

Çıktıyı incelediğimizde, dosya ismi, uzantısı ve bulunduğu dizin aynı olmasına rağmen hash değerinin 9D52B46F5DE41B73418F8E0DACEC5E9F olarak tamamen değiştiğini görüyoruz.

DİKKAT: Bu basit manipulasyon, salt hash tabanlı IOC avcılığının (threat hunting) neden yetersiz kalabileceğini kanıtlamaktadır. Savunma ekipleri orijinal D1A0... ile başlayan hash'i kara listeye almış olsa da, saldırgan sadece saniyeler süren `echo` komutu ile ürettiği yeni 9D52... hashli dosyayı sistemlere soktuğunda mevcut güvenlik kontrollerini (eğer sadece hash'e bakıyorlarsa) rahatlıkla atlatacaktır. Bu sebeple modern tehdit avcılığı sadece statik hash değerlerine değil; davranışsal analize (heuristics), bellek içi imzalara (in-memory signatures) ve YARA kurallarına dayanmak zorundadır.

IP Address (Kolay Seviye) - Pyramid of Pain ve Ağ Gizlenme Teknikleri

Siber güvenlikte bir ağa bağlı olan her cihazın (sunucular, son kullanıcı bilgisayarları, hatta IoT kapsamındaki güvenlik kameraları) benzersiz bir kimliği olmak zorundadır. Veri paketlerinin ağ üzerinde doğru kaynağa ve hedefe yönlendirilmesi tamamen **IP (Internet Protocol)** adresleri üzerinden gerçekleşir. Ancak bir tehdit avcısı veya olay müdahale (Incident Response) uzmanı için IP adresinin temel ağ işlevlerinden ziyade, saldırganın altyapısını deşifre etmek için kullanılan bir **IOC** (Indicator of Compromise - Uzlaşma Göstergesi) olması çok daha kritiktir.

Saldırgan altyapılarını ve savunma zorluklarını kategorize eden **Pyramid of Pain** (Acı Piramidi) modelinde, IP adresleri en alt katmanda yer alır ve "Yeşil" renk ile temsil edilir. Yeşil renk, "Trivial" (Çok Basit/Önemsiz) seviyesini ifade eder. Bu seviye, savunma ekipleri bir IP adresini tespit edip engellediğinde, saldırganın bu duruma adapte olup yeni bir IP adresiyle saldırısına devam etmesinin ne kadar "acısız" ve kolay olduğunu gösterir.

Defansif Perspektif: Çevre Güvenliği ve IP Bloklama

Mavi takım (Blue Team) açısından, bir tehdit aktörünün kullandığı zararlı IP adreslerini tespit etmek, olay müdahalesinin ilk adımlarından biridir. En yaygın ve temel savunma taktiği, tespit edilen bu zararlı IP adreslerinden gelen tüm **inbound** (içeri yönlü) istekleri kurumun **Perimeter Firewall** (Çevre Güvenlik Duvarı) üzerinde bloklamak, düşürmek (drop) veya reddetmektir (deny).

HTTP Requests		0	Connections		4	DNS Requests		4	Threats		0
Timeshift	Protocol	Rep	PID	Process name	CN	IP	Port				
85528 ms	TCP	⚠	1632	some_malicious_file.bi...	🇺🇸	50.87.136.52	443				
144.95 s	TCP	?	1632	some_malicious_file.bi...	🇩🇪	78.46.1.42	443				
205.35 s	TCP	⚠	1632	some_malicious_file.bi...	🇩🇪	134.119.253.108	443				
264.76 s	TCP	⚠	1632	some_malicious_file.bi...	🇺🇸	104.21.87.185	443				

ÖNEMLİ: Sandbox (Kum havuzu - Zararlı yazılımların izole ortamda çalıştırılıp incelendiği sistemler, örn: app.any.run) raporlarında veya Tehdit İstihbaratı (Threat Intelligence) akışlarında gördüğünüz zararlı IP adresleriyle asla kişisel veya kurumsal ağınızdaki üzerinden doğrudan etkileşime girmeyin. Bu tür bir etkileşim (ping atmak, port taraması yapmak veya tarayıcıdan gitmeye çalışmak) saldırganın sistemlerindeki loglara düşmenize ve kurumunuzun hedeflenmesine yol açabilir.

Ancak firewall seviyesinde yapılan IP tabanlı engellemeler günümüz sofistike saldırılarında kesin bir çözüm (bulletproof) sunmaz. Deneyimli bir saldırgan için bulut sağlayıcıları veya kiralık sunucular üzerinden yepyeni bir **Public IP** (Genel IP) adresi tahsis etmek ve altyapısını buraya taşımak sadece saniyeler sürer. Bu durum, savunma ekiplerini sürekli bir "köstebek vurmaca" (whack-a-mole) oyununun içine çeker.

Saldırganların Karşı Önlemi: Fast Flux (Hızlı Akış) Ağları

Saldırganlar, IP tabanlı engellemeleri sadece yeni bir IP olarak değil, bu süreci tamamen otomatize eden çok daha gelişmiş mimariler kurarak aşarlar. Mavi takımın IP bloklama stratejisini tamamen etkisiz hale getiren en popüler tekniklerden biri **Fast Flux** ağıdır.

Akamai'nin teknik analizlerine göre **Fast Flux**, **Botnet** (Bot ağları - Ele geçirilmiş ve saldırgan tarafından uzaktan kontrol edilen zombi bilgisayar orduları) operatörleri tarafından kullanılan bir **DNS** (Domain Name System - Alan Adı Sistemi) manipülasyon tekniğidir.

Fast Flux'ın Temel Amacı ve Çalışma Mantiği: Bir zararlı yazılım (malware) hedefe bulaştığında, talimat almak ve çaldığı verileri sızdırmak için saldırganın **C&C** (Command and

Control - Komuta ve Kontrol) sunucusu ile iletişim kurmak zorundadır. Güvenlik uzmanları bu iletişimi tespit edip engellemek ister. Saldırganlar, gerçek C&C sunucularını gizlemek için binlerce ele geçirilmiş cihazı (proxy/vekil sunucu olarak) C&C sunucusunun önüne bir kalkan gibi dizerler.

Teknik olarak Fast Flux, tek bir alan adının (domain name) arkasına yüzlerce veya binlerce farklı IP adresinin atanması ve bu IP adreslerinin sürekli, dakikalar hatta saniyeler içinde değiştirilmesi prensibine dayanır. Bu işlem, DNS kayıtlarındaki **TTL** (Time to Live - Yaşam Süresi) değerlerinin kasıtlı olarak çok düşük tutulmasıyla sağlanır.

Bir Fast Flux Senaryosunda Gerçekleşen Olay Örgüsü:

1. Kurum içindeki zararlı yazılım `malicious-domain.com` adresine bağlanmak için DNS sorgusu yapar.
2. DNS sunucusu, o an için aktif olan ele geçirilmiş bir cihazın (proxy) IP adresini döndürür (Örn: `198.51.100.45`).
3. Güvenlik cihazlarınız bu iletişimi yakalar ve `198.51.100.45` IP adresini firewall üzerinde bloklar.
4. Ancak DNS TTL süresi 60 saniye olduğu için, bir dakika sonra zararlı yazılım aynı alan adına gitmek istediğinde, DNS bu kez tamamen farklı, dünyanın başka bir yerindeki ele geçirilmiş bir cihazın IP adresini döndürür (Örn: `203.0.113.88`).
5. Firewall kuralınız artık işlevsizdir çünkü saldırı altyapısı zaten yer değiştirmiştir.

Bu sayede saldırganlar; ortalama (phishing) sitelerini, zararlı yazılım dağıtım sunucularını ve C&C iletişimlerini güvenlik profesyonellerinden, EDR (Endpoint Detection and Response) çözümlerinden ve hatta kolluk kuvvetlerinin sunucu kapatma (takedown) operasyonlarından korumuş olurlar. IP adreslerinin Pyramid of Pain'de neden en değersiz (yeşil) gösterge olduğu, Fast Flux gibi tekniklerin varlığı ile mükemmel bir şekilde kanıtlanmaktadır. Defansif açıdan sadece IP'ye odaklanmak yerine, DNS davranışlarına, DGA (Domain Generation Algorithm) anormalliklerine ve host üzerindeki davranışsal aktivitelere odaklanmak şarttır.

Alan Adları - Domain Names (Basit Seviye)

Pyramid of Pain (Acı Piramidi - Tehdit göstergelerinin saldırganlar için değiştirilme zorluğuna göre sınıflandırıldığı model) hiyerarşisinde bir üst basamağa, "Yeşil" (IP Adresleri) seviyesinden "Deniz Mavisi" (Teal) seviyesine yani **Domain Names** (Alan Adları) kategorisine geçiş yapıyoruz.

Teknik olarak bir alan adı, insanların akılda tutması zor olan IP adreslerini metinsel dizelere eşleyen bir haritalama sistemidir. Modern ağ mimarisinde bir alan adı; **Domain** (Örn: `evilcorp`) ve **TLD** (Top-Level Domain - Üst Düzey Alan Adı, Örn: `.com`) yapılarından oluşur. Daha spesifik hedefler için bu yapıya bir **Sub-domain** (Alt Alan Adı, Örn: `tryhackme.evilcorp.com`) eklenebilir. Ancak siber güvenlik ve olay müdahalesi (Incident Response) perspektifinden alan adları, sadece bir ağ yönlendirmesi değil, saldırganın altyapısının temel bir **IOC** (Indicator of Compromise - Uzlaşma Göstergesi) elemanıdır.

Defansif Perspektif: Saldırgan İçin Alan Adı Değiştirmenin Maliyeti

Acı Piramidinde yukarı çıkıldıkça saldırganın altyapısını değiştirmesi zorlaşır. IP adreslerini değiştirmek (Fast Flux gibi tekniklerle) saniyeler sürerken, bir alan adını değiştirmek saldırgan için nispeten daha zahmetlidir. Yeni bir alan adı kullanabilmek için saldırganın bu alan adını satın alması, kaydettirmesi (Registration) ve DNS (Domain Name System) kayıtlarını yapılandırması gerekir. Bu durum, savunma ekiplerine (Blue Team) zararlı alan adlarını tespit edip engellemek için değerli bir zaman penceresi sunar.

ÖNEMLİ: Ne yazık ki günümüzde DNS sağlayıcılarının gevşek güvenlik standartları ve sundukları otomatik API (Application Programming Interface) destekleri, saldırganların alan adı değiştirme sürecini büyük ölçüde hızlandırmıştır. Çoğu saldırgan, çalıntı kredi kartlarıyla veya kripto paralarla saniyeler içinde binlerce alan adını programatik olarak kaydedebilmektedir.

Homograph (Benzer Karakter) ve Punycode Saldırıları

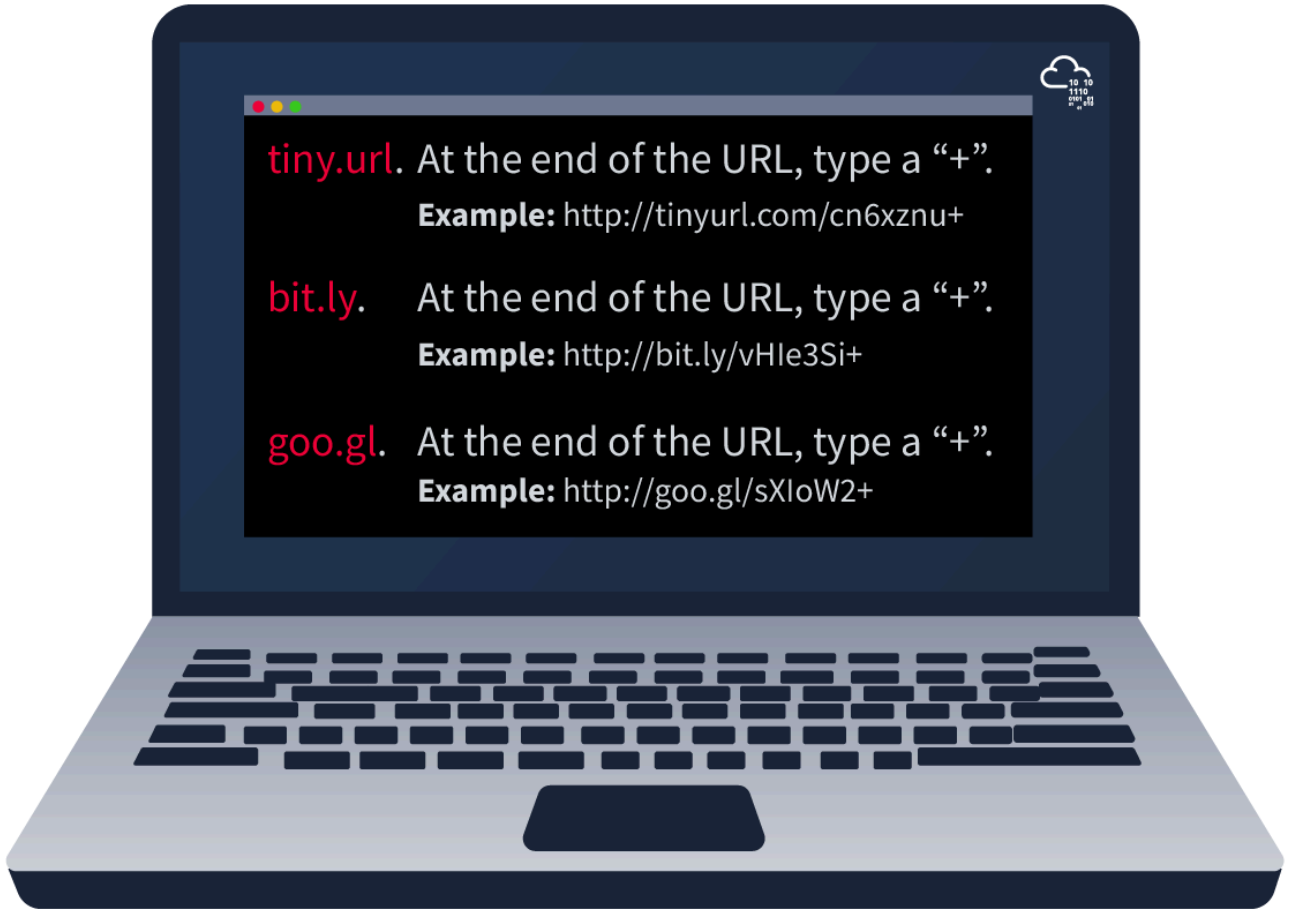
Saldırganların, kullanıcıları yasal görünümlü ancak zararlı **C2** (Command and Control - Komuta ve Kontrol) veya oltalama (Phishing) sunucularına yönlendirmek için kullandığı en sinsi tekniklerden biri **Punycode** saldırılarıdır.

Punycode tekniğinin temel amacı, ASCII (American Standard Code for Information Interchange) standart karakter setinde yazılamayan karakterleri (örneğin Türkçe, Kiril veya Yunanca harfler), Unicode ASCII kodlamasına dönüştürmektir. Saldırganlar bu mekanizmayı istismar ederek görsel olarak orijinaline birebir benzeyen sahte alan adları üretirler.

Örneğin, orijinal `adidas.de` alan adını taklit etmek isteyen bir saldırgan, Türkçe karakter olan "ı" harfini kullanarak `adıdas.de` şeklinde bir alan adı satın alabilir. Kurban bu bağlantıya tıkladığında, arka planda tarayıcı bu adresi gerçek bir Punycode URL'sine dönüştürür:

```
http://xn--addas-o4a.de/
```

Yukarıdaki kod bloğunda görülen `xn--` öneki, DNS sunucularına ve tarayıcılara bu alan adının bir Punycode olduğunu bildirir. Günümüzde Internet Explorer, Google Chrome, Microsoft Edge ve Apple Safari gibi modern tarayıcılar, bu tür **Obfuscation** (Gizleme/Karmaşıklıklaştırma) tekniklerini tespit edip adres çubuğunda gizlenmiş Punycode'un tam halini (yani `xn--` ile başlayan gerçek adresi) göstererek kullanıcıyı uyarma konusunda oldukça gelişmiştir.



URL Kısaltıcılar (URL Shorteners) ile Gizlenme

Saldırganların zararlı alan adlarını kurbanlardan ve güvenlik filtrelerinden saklamak için başvurdukları bir diğer yaygın yöntem **URL Shortener** (URL Kısaltıcı) servisleridir. Bu araçlar, uzun ve şüpheli görünen bir bağlantıyı (Örn: zararlı bir payload indirme adresi), güvenilir bir servisin alan adı üzerinden kısa ve benzersiz bir bağlantıya dönüştürür.

Saldırganların sıklıkla istismar ettiği popüler kısaltma servisleri arasında **bit.ly**, **goo.gl**, **ow.ly**, **s.id**, **smarturl.it**, **tiny.pl**, **tinyurl.com** ve **x.co** bulunur. Ortalama e-postalarında bu kısa linkleri gören kurban, arkasında hangi alan adının yattığını bilemez.

DİKKAT (Analiz Pratiği): Bir olay müdahalesi sırasında veya şüpheli bir e-postayı analiz ederken, kısaltılmış bir URL'nin kurbanı nereye yönlendirdiğini (HTTP 301 Redirect) öğrenmek istiyorsanız, linke ASLA doğrudan tıklamayın. Çoğu kısaltma servisi, URL'nin sonuna bir + (artı) işareti eklediğinizde, sizi hedefe yönlendirmek yerine bağlantının analiz ve istatistik sayfasına götürür.

Şüpheli Link: `http://bit.ly/ZararliLink`

Analiz Linki: `http://bit.ly/ZararliLink+`

Yukarıdaki işlemi yaptığınızda tarayıcınız, kısaltma servisinin veri tabanını sorgular ve zararlı sunucuyla (veya olası bir IP loglama mekanizmasıyla) hiçbir temas kurmadan gerçek hedef (Destination) URL'yi güvenli bir şekilde görmenizi sağlar. Bu tür gizlenmiş bağlantıları ağ

seviyesinde tespit etmek için **Proxy Logları** (Vekil Sunucu Kayıtları) ve **Web Server Logları** (Web Sunucusu Kayıtları) detaylı olarak incelenmelidir.

Any.run Sandbox Üzerinde Ağ ve Alan Adı Analizi

[Any.run](#), şüpheli dosyaları veya bağlantıları izole ve güvenli bir ortamda çalıştırarak (Detonation) davranışlarını analiz etmemizi sağlayan interaktif bir **Sandbox** (Kum Havuzu) servsidir. Bir zararlı yazılım çalıştırıldığında oluşturduğu tüm ağ trafiği, arayüzdeki sanal makine anlık görüntüsünün (snapshot) hemen altında bulunan "Networking" (Ağ) sekmesinden detaylıca incelenebilir.

ÖNEMLİ UYARI: Sandbox raporlarında tespit edilen IP adreslerine veya HTTP isteklerine kendi ağınızdan kesinlikle erişmeye çalışmayın. Bu adresler aktif bir zararlı yazılım altyapısının parçasıdır; doğrudan temas kurmanız, saldırıgana kurum IP adresinizi ifşa eder ve sistemlerinizi doğrudan hedef haline getirebilir.

Any.run üzerindeki ağ analizi üç ana kategoride incelenir ve her birinin arka planında yatan mantık şudur:

1. HTTP Requests (HTTP İstekleri)

Zararlı yazılım sanal ortamda tetiklendiği andan itibaren yaptığı tüm HTTP tabanlı bağlantıları gösterir. Bir tehdit avcısı bu logları okurken şunları arar:

- **Dropper (İndirici) Davranışı:** Sisteme ilk sızan küçük boyutlu yazılımın, asıl yıkıcı yükü (Payload veya Ransomware) dış bir web sunucusundan (GET isteğiyle) indirmeye çalışması.
- **Callback (Geri Arama):** Zararlı yazılımın, sisteme başarıyla yerleştiğini ve cihaz bilgilerini saldırıgana C2 sunucusuna (POST isteğiyle) iletmesi.

HTTP Requests	7	Connections	51	DNS Requests	20	Threats	0	Filter by PID, name or url	PCAP
Timeshift	Headers	Rep	PID	Process name	CN	URL	Content		
25853 ms	GET 204: No Content	✓	2572	chrome.exe		http://www.gstatic.com/generate_204	-		
44576 ms	GET 200: OK	✓	2572	chrome.exe		http://ctldl.windowsupdate.com/msdownload/update/v3/static/trusted/en/authrootstl.cab...	62.3 Kb + compressed		
76052 ms	HEAD 200: OK	✓	852	svchost.exe		http://edgedl.me.gvt1.com/edgedl/release2/chrome_component/YGkwa4MXjFWuSuERYWQY...	-		
81155 ms	GET 200: OK	✓	852	svchost.exe		http://edgedl.me.gvt1.com/edgedl/release2/chrome_component/YGkwa4MXjFWuSuERYWQY...	3.72 Kb + binary		
105.77 s	HEAD 200: OK	✓	852	svchost.exe		http://edgedl.me.gvt1.com/edgedl/release2/chrome_component/eua6zlfhpj3roq46nymxtbz...	3.72 Kb + crx		
105.77 s	GET 206: Partial Con...	✓	852	svchost.exe		http://edgedl.me.gvt1.com/edgedl/release2/chrome_component/eua6zlfhpj3roq46nymxtbz...	7.13 Kb + binary		
110.88 s	GET 206: Partial Con...	✓	852	svchost.exe		http://edgedl.me.gvt1.com/edgedl/release2/chrome_component/eua6zlfhpj3roq46nymxtbz...	3.11 Kb + binary		

2. Connections (Bağlantılar)

Zararlı yazılımın başka bir ana makineyle (Host) kurduğu tüm düşük seviyeli TCP/UDP bağlantılarını temsil eder. Burada şifrelenmemiş HTTP dışındaki trafiğe odaklanılır. Örneğin, saldırıgana kurban sistemden kritik verileri dışarı sızdırmak (Exfiltration) için başlattığı bir **FTP (File Transfer Protocol)** bağlantısı veya farklı bir port üzerinden kurulan kalıcı C2 tüneli bu sekmede tespit edilir.

	HTTP Requests	7	Connections	51	DNS Requests	20	Threats	0		Filter by PID, domain, name or ip	PCAP
	Timeshift	Protocol	Rep	PID	Process name	CN	IP	Port	Domain	ASN	Traffic
NETWORK	1309 ms	UDP	✓	4	System	?	192.168.100.255	138	-	-	↑ 2.21 Kb ↓ -
	1312 ms	UDP	✓	1076	svchost.exe	?	224.0.0.252	5355	-	-	↑ 48 b ↓ -
	1314 ms	UDP	✓	3740	svchost.exe	?	239.255.255.250	1900	-	-	↑ 1.41 Kb ↓ -
FILES	1315 ms	UDP	✓	4	System	?	192.168.100.255	137	-	-	↑ 1.70 Kb ↓ -
	4397 ms	UDP	✓	1076	svchost.exe	?	224.0.0.252	5355	-	-	↑ 44 b ↓ -
	4399 ms	UDP	✓	1076	svchost.exe	?	224.0.0.252	5355	-	-	↑ 48 b ↓ -
DEBUG	4405 ms	UDP	✓	2044	chrome.exe	?	239.255.255.250	1900	-	-	↑ 696 b ↓ -
	4408 ms	UDP	✓	1076	svchost.exe	?	224.0.0.252	5355	-	-	↑ 44 b ↓ -
	5412 ms	TCP	✓	2572	chrome.exe	🇺🇸	142.250.185.173	443	accounts.google.com	GOOGLE	No Data
	5573 ms	TCP	✓	2572	chrome.exe	🇺🇸	142.250.186.142	443	clients2.google.com	GOOGLE	↑ 1.02 Kb ↓ 8.64 Kb

3. DNS Requests (DNS İstekleri)

Sandbox analizi sırasında yapılan tüm alan adı çözümleme isteklerini (DNS Queries) gösterir. Gelişmiş zararlı yazılımlar, analiz ortamlarını atlarmak (Evasion) için DNS isteklerini bir silah olarak kullanır.

	HTTP Requests	7	Connections	51	DNS Requests	20	Threats	0		Filter by IP or domain	PCAP
	Timeshift	Status	Rep	Domain	IP						
NETWORK	5371 ms	Responded	?	ice-eng.app.box.com	74.112.186.144						
	5373 ms	Responded	✗	accounts.google.com	142.250.185.173						
	5373 ms	Responded	✓	clients2.google.com	142.250.186.142						
FILES	5374 ms	Responded	✓	clients2.googleusercontent.com	142.250.186.97						
	11478 ms	Responded	✓	ssl.gstatic.com	142.250.184.227						
	25794 ms	Responded	✓	www.gstatic.com	142.250.186.99						
DEBUG	27799 ms	Responded	✓	cdn01.boxcdn.net	104.16.74.20						
					104.18.103.56						
					52.222.206.6						
	29500 ms	Responded	✓	cdn.amplitude.com	52.222.206.178						

- **Connectivity Check (Bağlantı Kontrolü):** Zararlı yazılım, çalışmaya başlamadan önce google.com veya kendi C2 sunucusu gibi adreslere DNS isteği yollar. Eğer internete çıkamadığını veya sahte DNS yanıtları aldığını (ki çoğu lokal sandbox sahte yanıtlar üretir) fark ederse, bir analiz laboratuvarında olduğunu anlar ve kendisini sonlandırır (Terminate). Bu sekme, zararlı yazılımın hangi alan adlarıyla iletişim kurmaya programlandığını (Hardcoded Domains) veya hangi algoritmaları (DGA - Domain Generation Algorithm) kullandığını tespit etmek için en kritik alandır.

Host Artifacts (Uç Nokta Kalıntıları) - Orta Seviye

Pyramid of Pain (Acı Piramidi) üzerinde bir adım daha yukarı çıkarak "Sarı" bölgeye, yani **Host Artifacts** (Uç Nokta Kalıntıları / Ana Bilgisayar Eserleri) seviyesine ulaşıyoruz.

Ağ göstergelerinden (IP adresleri ve Alan adları) farklı olarak, bu seviyede bir tespitle karşılaştığında saldırganın canı biraz daha fazla sıkılacaktır. Çünkü savunma ekipleri bir "Host Artifact" tespit edip engellediğinde, saldırganın saldırı araçlarını, sisteme sızma metodolojilerini veya zararlı yazılımının çalışma mantığını değiştirmek için geri dönmesi gerekir. Bu süreç saldırgan için son derece zaman alıcıdır ve yeni araçlar geliştirmek için ekstra efor/kaynak harcamasını gerektirir.

Host Artifact (Kalıntı) Nedir?

Host Artifact'ler, bir saldırganın hedef sistem (sunucu veya son kullanıcı bilgisayarı) üzerinde çalışırken veya zararlı yazılımını koştururken arkasında bıraktığı **izler ve somut kanıtlardır**. Bunlara şunlar dahildir:

- Değiştirilen veya yeni eklenen **Registry (Kayıt Defteri)** değerleri.
- Şüpheli ve anormal **Process (İşlem)** hiyerarşileri (Örn: Bir ofis programının komut satırı açması).
- Zararlı uygulamalar tarafından diske yazılan (dropped) veya değiştirilen dosyalar.
- Saldırı kalıpları veya o tehdide özgü herhangi bir IOC (Uzlaşma Göstergesi).

Görsel Analizi: Bir Saldırının Anatomisi

Paylaştığınız ekran görüntüleri, bir tehdit avcısının (Threat Hunter) veya olay müdahale uzmanının EDR (Endpoint Detection and Response) sistemlerinde veya Sandbox raporlarında tam olarak ne aradığını mükemmel bir şekilde özetliyor. Gelin bu izleri adım adım inceleyelim:

1. Şüpheli İşlem Yürütmesi (Suspicious Process Execution)

Normal şartlarda Microsoft Word (WINWORD.EXE) bir döküman okuyucusudur. Ancak ilk görselde, Word uygulamasının (PID: 3640) altında api-ms-win-downlevel-user32-l1-... (PID: 3300) adında, "EffectDemo MFC Application" tanımlı şüpheli bir alt işlem (child process) başlattığını görüyoruz.

WINWORD.EXE	0.01	51,500 K	134,300 K	3640 Microsoft Word	Microsoft Corporation
api-ms-win-downlevel-user32-l1-...		4,632 K	11,192 K	3300 EffectDemo MFC Application	

Bir ofis programının bu tarz harici ve bilinmeyen çalıştırılabilir dosyaları tetiklemesi, genellikle döküman içine gizlenmiş bir makronun (VBA) veya zafiyet istismarının (Exploit) çalıştığının en büyük göstergesidir.

2. Şüpheli Ağ ve Dosya Olayları (Suspicious Events)

Sistemde asıl yıkıcı faaliyetlerin POWERSHELL.exe (PID: 3540) üzerinden yürütüldüğünü görüyoruz:

- **Ağ Trafiği:** PowerShell, 35.214.215.33 IP adresine 80 portu (HTTP) üzerinden sürekli olarak TCP bağlantıları kuruyor (TCP Receive). Bu büyük ihtimalle saldırganın Komuta ve Kontrol (C2) sunucusu veya bir zararlı yazılım indirme noktasıdır.
- **Dosya Yazma (WriteFile):** PowerShell ağdan aldığı bu verileri anında diske yazıyor. Yazdığı dizin ve dosya oldukça şüpheli:
C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe .

Time o...	Process Name	PID	Operation	Path	Result
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS
324.26...	Powershell.exe	3540	WriteFile	C:\Users\RussianPanda\Jehhzda\Ben14fr\G_jugk.exe	SUCCESS
324.26...	Powershell.exe	3540	TCP Receive	192.168.75.222:1047 -> 35.214.215.33.80	SUCCESS

3. Diske Bırakılan Dosyalar (Dropped Files) ve Hash Analizi

İkinci görsel, saldırganın sistemde bıraktığı dosyaların (Artifacts) bir özetini sunuyor ve daha önceki "Hash Değerleri" konumuzla doğrudan bağlantı kuruyor:

- WINWORD.EXE , geçici (Temp) dizine .exd ve .doc uzantılı dosyalar bırakmış.
- Önceki adımda gördüğümüz POWERSHELL.exe , G_jugk.exe isimli zararlı dosyayı (MD5: 92F58C4E...) sisteme indirmiş.
- **Kritik Detay:** G_jugk.exe dosyası çalıştıktan sonra (PID: 1640), sistemin başka bir konumuna (AppData\Local\photowiz\) regidle.exe adında yeni bir dosya bırakmış. İşin ilginç yanı, G_jugk.exe ve regidle.exe dosyalarının MD5 ve SHA256 Hash değerleri birbiriyle tamamen aynı!

2728	WINWORD.EXE	C:\Users\admin\AppData\Local\Temp\VBEMSFMSForms.exd	MD5: CC118FD14D6ECC83477B69FF06C6C587	SHA256: A4E8F5821887AC26449C33D9B027CE31BE0E7203DD035C5DC7D34A9AEF01A6DA	tlb
2728	WINWORD.EXE	C:\Users\admin\AppData\Local\Temp\~\$0-100120 CDW-102220.doc	MD5: 2E7A3442236F2D50C669BC791888BD69	SHA256: BF007001BACF8F6ABF371B0B2797B7D13B741879E1E5B76FB616A934318418A9	pgc
3828	Powershell.exe	C:\Users\admin\Jehhzda\Ben14fr\G_jugk.exe	MD5: 92F58C4E2F524EC53EBE10D914D96CCB	SHA256: 4A9E32BC5348265C43945ADAAAF140B98B64329BD05878BC13671FA916F423710	executable
1640	G_jugk.exe	C:\Users\admin\AppData\Local\photowiz\regidle.exe	MD5: 92F58C4E2F524EC53EBE10D914D96CCB	SHA256: 4A9E32BC5348265C43945ADAAAF140B98B64329BD05878BC13671FA916F423710	executable

Önceki derslerimizde iki dosyanın aynı Hash değerine sahip olmasının ne anlama geldiğini konuşmuştuk. Sizce saldırgan neden G_jugk.exe isimli dosyasını, sistemin başka bir köşesine regidle.exe ismiyle ama birebir aynı içerikle (aynı hash ile) kopyalama ihtiyacı hissetmiş olabilir?

Ağ Kalıntıları (Network Artifacts) - Can Sıkıcı (Annoying) Seviye

Pyramid of Pain (Acı Piramidi) hiyerarşisinde "Sarı" bölgede yer alan bir diğer kritik kategori **Network Artifacts** (Ağ Kalıntıları) seviyesidir.

Uç Nokta Kalıntıları (Host Artifacts) ile aynı zorluk derecesini paylaşan bu seviyede bir tespitle karşılaşp engellendiğinde, saldırganın durumu **"Can Sıkıcı" (Annoying)** bulacaktır. Çünkü saldırgan, mevcut altyapısını, iletişim taktiklerini veya zararlı yazılım araçlarını yeniden yapılandırmak için zaman ve efor harcamak zorunda kalır. Bu gecikme, mavi takıma (savunma ekiplerine) yaklaşan tehditleri tespit etmek veya mevcut sızıntıları temizlemek (remediation) için altın değerinde bir zaman penceresi kazandırır.

Ağ Kalıntısı (Network Artifact) Nedir?

Ağ kalıntıları, saldırganın hedef sistemle veya kendi Komuta-Kontrol (C2) sunucusuyla iletişim kurarken ağ trafiğinde bıraktığı ayırt edici izlerdir. En yaygın örnekleri şunlardır:

192.168.100.140	194.187.133.160	936 HTTP	POST /Nqd1z/w2BG/ HTTP/1.1
192.168.100.140	98.174.164.72	936 HTTP	POST /ghMuzyNcNwN/kMmYdVithxeVy/o2Feo8eu7Jyv/O2M8WIf9SpyCp/yLVEV96e0syd5URJ477/8wdGXdz9k9hhJjWp/ HTTP/1.1
192.168.100.140	103.86.49.11	936 HTTP	POST /VCv0qXMjgEehauu/AyEp/O9Qn2/R6Rj7Gw9e0v6yJ/fC5a36YfopGe/Q2AwYvSohZiyaEtbb0/ HTTP/1.1
192.168.100.140	78.24.219.147	904 HTTP	POST /jC0c/oQQPmafJlpMi6n3/Pba0/K7oB22aAUKQ61A6r/Go0MY/ HTTP/1.1
192.168.100.140	50.245.107.73	888 HTTP	POST /ukXcIs1jsvd7W/h2VQ1YqB/csuQkgUq1kakMvQRJ9/NCjJodG/ HTTP/1.1
192.168.100.140	110.145.77.103	888 HTTP	POST /QZvVQ6o1I/DYk9QgXU/HtoXMcRhbYcJhgami/5NsCejn3/ HTTP/1.1

- **User-Agent (Kullanıcı Aracısı) Dizeleri:** İnternete çıkan her uygulama (tarayıcılar, betikler, zararlı yazılımlar), HTTP istek başlığında kendini tanıtan bir "User-Agent" dizesi kullanır (RFC2616 standardı). Saldırganlar bazen kurumunuzda daha önce hiç görülmemiş, sıradışı veya zararlı yazılıma özgü (hardcoded) özel User-Agent dizeleri kullanabilirler.
- **C2 (Komuta ve Kontrol) Bilgileri:** Zararlı yazılımın evine dönerken kullandığı spesifik portlar, sertifika bilgileri veya bağlantı periyotları (Beaconing).
- **URI Kalıpları:** Özellikle şüpheli HTTP POST isteklerinde karşılaşılan benzersiz dizin yolları veya parametre yapıları (Örn: /Nqd1z/w2BG/ veya /VCv0q...).

Tespit ve Analiz Metodolojisi

Ağ kalıntılarını tespit etmek için genellikle iki temel kaynaktan faydalanılır:

1. **PCAP (Packet Capture) Dosyaları:** Ağ paket dökümlerini içeren dosyalardır. **Wireshark** (grafik arayüzlü) veya **TShark** (komut satırı tabanlı) gibi araçlarla derinlemesine analiz edilir.
2. **IDS (Saldırı Tespit Sistemi) Logları:** **Snort** gibi sistemlerin, imza tabanlı kurallarına takılan şüpheli ağ anormalliklerinin ürettiği alarmlardır.

Görsel Analizi ve TShark Kullanımı

Paylaştığınız ilk görseldeki yeşil blokta, bir zararlı yazılımın art arda yaptığı şüpheli HTTP POST isteklerinin hedef URI'lerini görüyoruz. İlk bakışta bu rastgele harflerden oluşan (örneğin /ukXcI...) istekleri belirli bir zararlı yazılıma bağlamak (Attribution) zordur.

Bu noktada ağ göstergelerini (Indicator) daha derinlemesine kazmak için **TShark** devreye girer. Görseldeki komutu parçalarına ayırırsak:

[illegible]

```
tshark -Y http.request -T fields -e http.host -e http.user_agent -r
analysis.pcap
```

- `tshark` : Wireshark'ın terminal versiyonudur.
- `-Y http.request` : Display Filter (Görüntüleme Filtresi) uygular. Tüm pcap dosyasındaki sadece HTTP isteklerini ayıklar.
- `-T fields` : Çıktının sadece belirli alanlarını (fields) tablo/sütun formatında göstermesini söyler.
- `-e http.host -e http.user_agent` : Ekrana yazdırılacak spesifik alanları seçer. (Hedef Host ve User-Agent).
- `-r analysis.pcap` : Okunacak (read) hedef ağ kayıt dosyasını belirtir.

Sonuç ve Savunma Stratejisi: TShark komutunun çıktısında (ikinci görsel), bağlantı kuran kaynağın sürekli olarak aynı, uzun ve spesifik User-Agent dizesini Mozilla/4.0 (compatible; MSIE 7.0; Windows NT 6.1... kullandığı görülüyor. Bu, **Emotet** gibi bilinen Trojan'ların (Truva Atı) kullandığı karakteristik bir User-Agent'tır.

Eğer bir güvenlik analisti olarak bu özel User-Agent dizesini tespit ederseniz, Güvenlik Duvarı (Firewall) veya Proxy seviyesinde bu User-Agent'ı içeren tüm trafiği doğrudan **bloklayabilirsiniz**. Bunu yaptığınız an, hedef ağınızdaki tüm zararlı yazılımların saldırganla iletişimi kesilir. Saldırganın bu engeli aşabilmesi için zararlı yazılımının kaynak koduna dönüp, User-Agent yapısını değiştirmesi ve zararlı yazılımını tekrar derlemesi gerekecektir ki bu da saldırgan için oldukça "Can Sıkıcı" bir süreçtir.

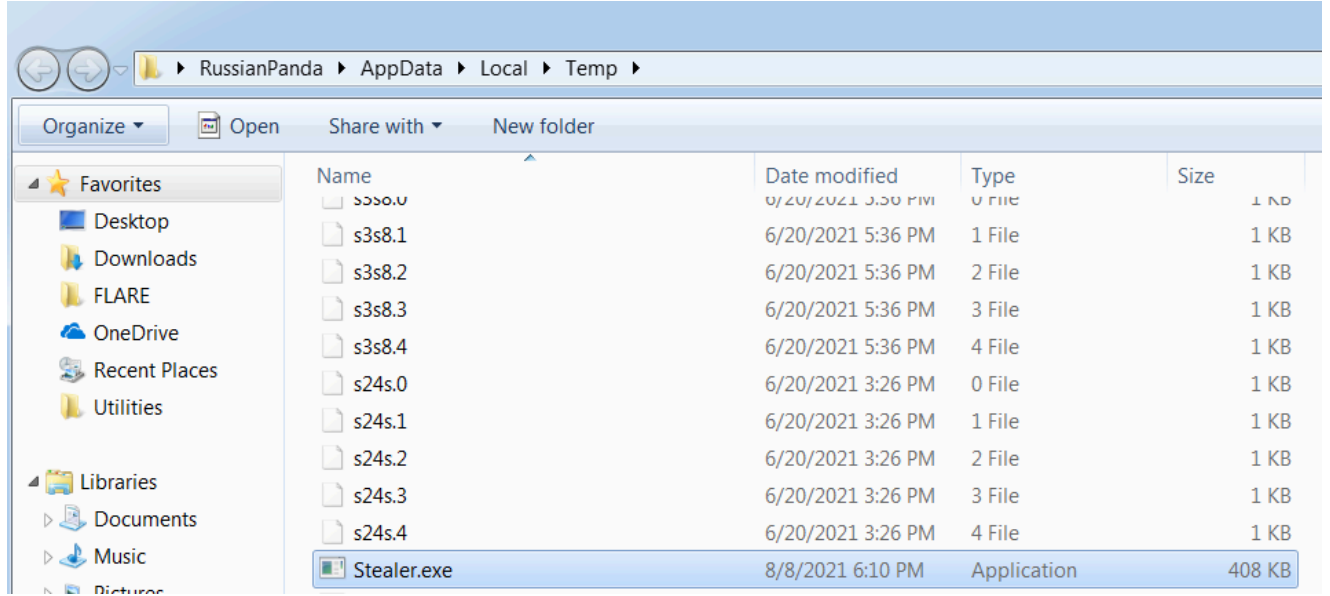
Araçlar (Tools) - Zorlu (Challenging) Seviye

Bu aşamada sadece bıraktıkları izleri (Artifacts) değil, doğrudan saldırıyı gerçekleştirmek için kullandıkları **Araçları (Tools)** tespit edip engelliyoruz. Bir saldırganın özel olarak geliştirdiği bir ortalama makrosunu (maldoc), ağınıza sızmak için yazdığı özel bir arka kapıyı (backdoor), C2 iletişim aracını veya özel derlenmiş bir .EXE / .DLL dosyasını tespit edip blokladığınızı düşünün.

Bu senaryoda saldırganın sadece taktik değiştirmesi yetmez; cebinden para harcayarak yeni bir araç satın alması, haftalarını harcayarak sıfırdan yeni bir zararlı yazılım kodlaması veya yeni bir aracı kullanmayı öğrenmek için eğitime girmesi gerekir. Savunma ekipleri için bu, kelimenin tam anlamıyla **"Oyun Bitti" (Game Over)** seviyesine yakın bir zaferdir.

Zararlı Aracın (Tool) Sisteme İnmesi ve Çalışması

İlk görselde, bir saldırgan aracının (Trojan) hedef sistemde nasıl konumlandığını görüyoruz:



- **Dosya Bırakma (Dropping):** Trojan, C:\Users\RussianPanda\AppData\Local\Temp dizinine Stealer.exe (Bilgi çalıcı bir zararlı yazılım aracı) adında 408 KB boyutunda şüpheli bir dosya bırakmış. Temp (Geçici) dizinler, sistem tarafından sık sık temizlense de, yazma izni gerektirmediği için zararlı yazılımların ilk durağıdır.
- **Çalıştırma (Execution):** İkinci aşamada, payload.exe (PID: 1356) isimli ana aracın, az önce diske bırakılan Stealer.exe (PID: 2928) aracını tetikleyerek belleğe (RAM) yüklediğini (11.63 MB) görüyoruz.

payload.exe	1356	12.09 MB	WIN-31...\RussianPanda
Stealer.exe	2928	11.63 MB	WIN-31...\RussianPanda Galactus

Bu araçları statik hash'ler veya IP'ler yerine "davranışsal veya yapısal" olarak tespit edebilirsek, saldırganın tüm operasyonunu çökertebiliriz.

Bu Seviyedeki Savunma Silahlarımız

Saldırganların araçlarını tespit etmek için güvenlik analistlerinin (Threat Hunters) kullandığı çok daha gelişmiş yöntemler vardır:

- **YARA Kuralları:** Zararlı yazılımların "İsviçre Çakısı" olarak bilinir. Bir dosyanın içindeki belirli metin dizelerini, hexadecimal (onaltılık) kalıpları veya API çağrılarını arayan özel kurallar yazmanızı sağlar. Saldırgan aracının kodunu değiştirse bile, kullandığı temel kütüphaneleri YARA ile yakalayabilirsiniz.
- **Antivirüs (AV) İmzaları ve Davranışsal Analiz:** Sadece dosyanın ne olduğuna değil, "ne yapmaya çalıştığına" (örneğin izinsiz kamera açma veya registry silme) odaklanan tespit mekanizmaları.
- **Tehdit İstihbaratı Platformları:** MalwareBazaar, Malshare veya SOC Prime gibi platformlar, dünyadaki diğer güvenlik uzmanlarının keşfettiği yeni saldırı araçlarının imzalarını ve tespit kurallarını paylaştığı merkezlerdir.

Kriptografik Hash'in Zayıflığı ve Yeni Silahımız: Fuzzy Hashing (SSDeep)

Önceki aşamalarda MD5 ve SHA-256 gibi kriptografik özetleme (Hashing) algoritmalarını işlemiştik ve en büyük zafiyetlerinin "**Çığ Etkisi**" (**Avalanche Effect**) olduğunu belirtmiştik: Dosyanın içindeki tek bir bit bile değişse, Hash değeri tamamen değişiyordu. Bu, saldırganların araçlarını (Tools) ufak tefek değişikliklerle gizlemesini kolaylaştırıyordu.

İşte tam bu noktada **Fuzzy Hashing (Bulanık Özetleme)** devreye girer. En popüler örneği **SSDeep**'tir.

DETECTION	DETAILS	RELATIONS	BEHAVIOR	COMMUNITY	13 +
Basic Properties ⓘ					
MD5	9498ff82a64ff445398c8426ed63ea5b				
SHA-1	36f9ca40b3ce96fcee1cf1d4a7222935536fd25b				
SHA-256	8b2e701e91101955c73865589a4c72999aeabc11043f712e05fdb1c17c4ab19a				
Vhash	025056657d755510804011z9005b9z25z12z3afz				
Authentihash	ad56160b465f7bd1e7568640397f01fc4f8819ce6f0c1415690ecee646464cec				
Imphash	d7584447a5c5ca9b4a55946317137951				
Rich PE header hash	fa4dbca9180170710b3c245464efa483				
SSDEEP	6144:Gz90qLc1zR98hUb4UdjzEwG+vqAWiR4EXePbix67CNzjX:Gz90qLc1lWhUbhVqJPbiQ7CNzb				
TLSH	T1DB44CF267660D833D0DF94316C75C3F9673BFC2123215A6B6A4417699E307E0AE7839E				
File type	Win32 EXE				
Magic	PE32 executable for MS Windows (GUI) Intel 80386 32-bit				
TrID	Win32 Executable MS Visual C++ (generic) (48.8%)				
TrID	Win64 Executable (generic) (16.4%)				
TrID	Win32 Dynamic Link Library (generic) (10.2%)				
TrID	Win16 NE executable (generic) (7.8%)				
TrID	Win32 Executable (generic) (7%)				
File size	249.00 KB (254976 bytes)				

SSDeep Nasıl Çalışır? SSDeep, dosyanın tamamına tek bir hash atamak yerine, dosyayı küçük bloklara (parçalara) böler ve her blok için ayrı bir hash hesaplayarak bunları birleştirir.

Sonuç: Eğer saldırgan `Stealer.exe` aracının içine zararsız bir virgül eklerse veya dosyanın sadece %5'ini değiştirirse, MD5 tamamen değişecektir ancak **SSDeep değeri orijinaliyle %95 oranında eşleşecektir!** İkinci görseldeki VirusTotal ekranında yer alan SSDeep dizesine (6144:Gz90qLc1zR...) dikkat edin. Güvenlik uzmanları bu dizeleri kullanarak "Benzerlik Analizi" (Similarity Analysis) yaparlar. Saldırgan aynı aracın farklı versiyonlarını derleyip gönderse bile, Fuzzy Hashing sayesinde analistler *"Bu dosya daha önce gördüğümüz Stealer aracıyla %88 aynı kod yapısına sahip"* diyerek aracı anında tespit edip imha edebilirler.

Saldırganın kullanacağı ağ altyapısını engelledik, geride bıraktığı izleri temizledik ve en son olarak geliştirdiği aracı (Tool) da Fuzzy Hashing ve YARA ile kullanılamaz hale getirdik. Geriye sadece bir adım kaldı.

Sizce bir saldırganın "Kullandığı Aracı" engellemekten daha zorlu ve onlara en çok acı verecek olan, piramidin zirvesindeki (TTPs) o son darbe ne olabilir?